

ERP Client Project Setup Guide

Step 1 — Go to Root Project

```
cd erp-system
```

Step 2 — Create Next.js Client

```
npx create-next-app@latest client
```

Step 3 — Recommended Answers

- ✓ TypeScript → Yes
 - ✓ ESLint → Yes
 - ✓ Tailwind CSS → Yes
 - ✓ src/ directory → Yes
 - ✓ App Router → Yes
 - ✓ Turbopack → Yes
 - ✓ Import Alias → Yes
-

Step 4 — Go Inside Client

```
cd client
```

Step 5 — Install Important Packages

API + State Management

```
npm install axios @tanstack/react-query zustand
```

Form + Validation

```
npm install react-hook-form zod @hookform/resolvers
```

UI Packages

```
npm install lucide-react sonner
```

Authentication Helpers

```
npm install js-cookie
```

Step 6 — Install ShadCN UI

```
npx shadcn@latest init
```

Step 7 — Recommended Folder Structure

```
client/  
├─ src/  
│  
│   ├─ app/  
│   └─ components/
```

```
|   ├── modules/
|   ├── services/
|   ├── hooks/
|   ├── store/
|   ├── lib/
|   ├── types/
|   ├── constants/
|   ├── providers/
|   └── utils/
|
|── public/
|── package.json
|── tsconfig.json
```

Step 8 — Create Core Folders

```
mkdir src/components
mkdir src/modules
mkdir src/services
mkdir src/hooks
mkdir src/store
mkdir src/lib
mkdir src/types
mkdir src/constants
mkdir src/providers
mkdir src/Utils
```

Step 9 — Create Axios Instance

src/lib/axios.ts

```
import axios from "axios"

const axiosInstance = axios.create({
  baseURL: "http://localhost:5000/api/v1",
  withCredentials: true
})
```

```
export default axiosInstance
```

Step 10 — Setup React Query Provider

src/providers/query-provider.tsx

```
"use client"

import { QueryClient, QueryClientProvider } from "@tanstack/react-query"
import { ReactNode } from "react"

const queryClient = new QueryClient()

const QueryProvider = ({ children }: { children: ReactNode }) => {
  return (
    <QueryClientProvider client={queryClient}>
      {children}
    </QueryClientProvider>
  )
}

export default QueryProvider
```

Step 11 — Setup Root Layout

src/app/layout.tsx

```
import type { Metadata } from "next"
import "../globals.css"

import QueryProvider from "@/providers/query-provider"

export const metadata: Metadata = {
  title: "ERP System",
  description: "ERP Dashboard"
}
```

```
export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode
}>) {
  return (
    <html lang="en">
      <body>
        <QueryProvider>
          {children}
        </QueryProvider>
      </body>
    </html>
  )
}
```

Step 12 — Create Home Page

src/app/page.tsx

```
export default function Home() {
  return (
    <div>
      <h1>ERP Frontend Running</h1>
    </div>
  )
}
```

Step 13 — Run Client

```
npm run dev
```

Step 14 — Open Browser

```
http://localhost:3000
```

Expected Output

ERP Frontend Running

Recommended ERP Client Modules

```
modules/  
├─ auth/  
├─ dashboard/  
├─ employee/  
├─ inventory/  
├─ sales/  
├─ purchase/  
├─ invoice/  
└─ analytics/
```

Recommended Learning Order

1. Login UI
 2. Auth Integration
 3. Dashboard
 4. Employee Module
 5. Inventory Module
 6. Sales Module
 7. Charts & Analytics
 8. Role Based UI
 9. Notifications
 10. Dark Mode
-

Recommended Stack

Technology	Usage
Next.js	Frontend Framework
TypeScript	Type Safety

Technology	Usage
Tailwind CSS	Styling
ShadCN UI	Professional Components
React Query	API State Management
Zustand	Global State
Axios	API Calls
Zod	Validation

Final Goal

Build:

- ERP Dashboard
- Inventory Management
- Sales Management
- Employee Management
- Analytics Dashboard
- Real-time Notifications

This becomes a real full-stack enterprise ERP project.